

The discipline “Object-Oriented Programming (OOP): Java”

Lecture 1: Introduction to Object-Oriented Programming (OOP) and
Functional Programming (FP)

Where is Java used?

Examples of popular applications and systems



Java is a universal programming language. It is used in many different areas: from mobile applications and games to servers of large internet companies.



Java – a programming language.

JVM – a platform for running programs.

Spring – a framework for fast development of Java applications.

NETFLIX

Netflix
Streaming service



Java is used in the server infrastructure of Netflix to process millions of user requests. A modern Netflix stack also includes **Spring Boot**.

What does Java do here?

- Processing user requests
- Recommendation systems
- Billing, analytics and monitoring
- Working with large amounts of data

Uber

Uber
Ride-hailing service



Java is used in many backend services of Uber, including route search, maps, logistics and payments.

What does Java do here?

- Maps and geolocation
- Route search and trip matching
- Payments and transactions
- Security and user authorization



Android
Mobile platform



Java was historically one of the main languages for Android. Today, Google recommends Kotlin, but Android still fully supports **Java**.

What does Java do in Android?

- Application logic
- Working with the user interface
- Background services
- Access to device features (camera, GPS, network, etc.)

MINECRAFT

Minecraft
Java Edition
Computer game

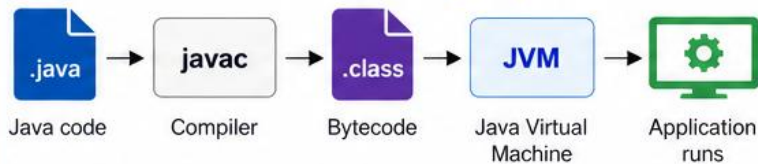


Minecraft: Java Edition is completely written in **Java**. It is one of the most famous examples of a real application/game built with **Java**.

What does Java do here?

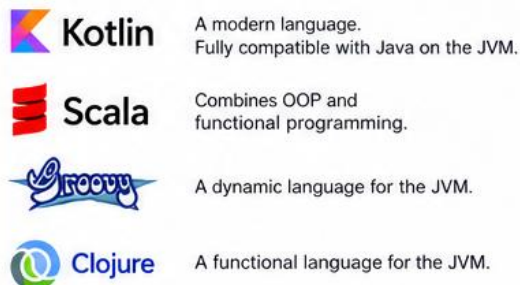
- Game logic
- World generation
- Physics and interaction of objects
- Multiplayer networking

How Java works in large systems

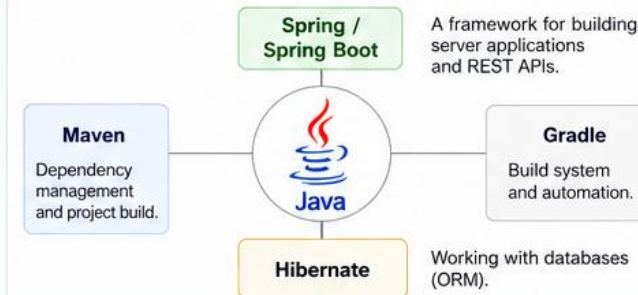


The same bytecode (.class) can run on different operating systems: Windows, Linux, macOS, etc.

Similar languages and technologies on the JVM



Java development ecosystem



Conclusion: **Java** is widely used in large internet services, mobile applications, games and corporate systems. It is a reliable language for building scalable and high-performance applications.

3

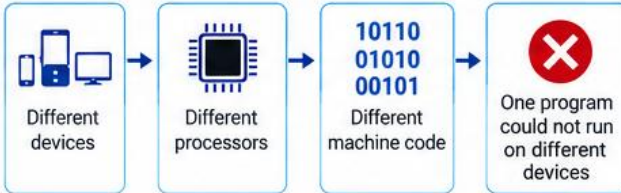
Why did Java appear?

The problem at the beginning of the 1990s

In the early 1990s, electronic devices became common and more powerful:

- televisions
- personal computers
- set-top boxes
- home appliances
- and others.

However, each device used different processors and operating systems.



GOAL
Sun Microsystems wanted to create a software platform that could run on different devices.
✓ Portable ✓ Reliable ✓ Secure ✓ Independent of hardware

Main idea:
One program — many devices.

3

From C++ to Oak

The story of a new language

At first, the team considered using C++, but it was too complex for the needs of the project. The new language had to be:

- simple
- safe
- reliable
- portable

So the Green Project team started to develop a new programming language.

1991 — OAK

The new language was initially called Oak.

The name came from an oak tree that grew outside the office window.



1992 — Star7

Oak was demonstrated in an experimental device called Star7.

Star7 was a portable controller for interacting with electronic devices.



Star7 (1992)



Java was originally created for the Internet. It was developed for consumer electronic devices.

2

Who created Java?

The Green Project at Sun Microsystems

In 1991, at Sun Microsystems, the Green Project research team started to work on a new programming language for electronic devices.

A small team of engineers worked on the idea of a portable and secure language.



James Gosling

- Created the Java programming language
- Known as the "father of Java"



Patrick Naughton

- One of the key project members
- Worked on the language implementation and system architecture



Mike Sheridan

- Helped with the project and its development
- Worked on business strategy and product direction

THE GREEN TEAM



4

From Oak to Java

From electronic devices to the Internet

The original goal was to develop software for consumer electronic devices.

But then the Internet (World Wide Web) started to grow rapidly.

The team realized that Java could also be used on the Internet.



Main idea: A Java program should not depend on one computer. It should run on different computers and platforms.

KEY IDEA OF JAVA

WRITE ONCE, RUN ANYWHERE

Write a program one time — and run it on different platforms.



Next: How can one Java program run on different computers?
(Java → Bytecode → JVM → Different operating systems)

Lecture 1. Introduction to OOP and FP

Welcome to the world of modern programming with Java!

In this lecture, we will get familiar with two main programming paradigms — the object-oriented approach (OOP) and the functional approach (FP), and we will take the first steps in learning the Java language.



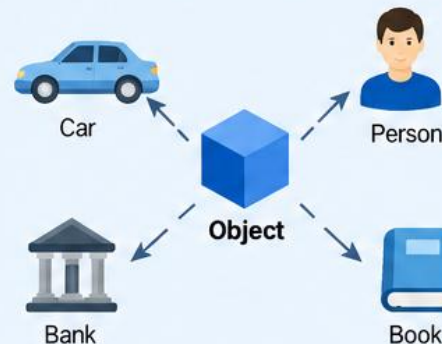
Goals of this lecture:

- ✓ understand the key ideas of OOP and FP
- ✓ learn what the Java language is and how it works
- ✓ write and run a simple Java application
- ✓ get familiar with the basics of classes, objects and references
- ✓ learn to input and output data in a console application



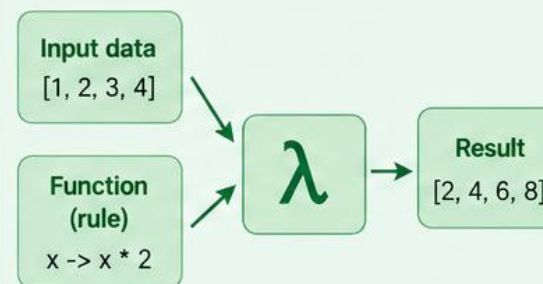
Java — is a modern, secure and platform-independent programming language, widely used to create different applications: from mobile and web applications to corporate systems.

OOP – the world of objects



A program is an interaction of objects, each of which has a state (data) and behavior (methods).

FP – calculations through functions



Main ideas: functions as values, immutable data, composition of functions, pure functions.

Here we have input data. and we have a function. The function takes the input data. As a result, we get the output value.

Why choose Java?



Reliability and security



Platform independence ("Write Once, Run Anywhere")



High performance



Large community and ecosystem



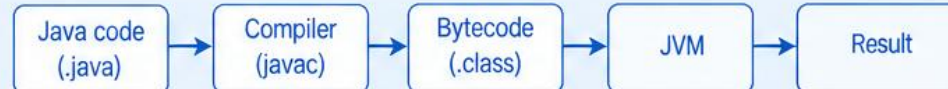
Rich libraries and tools

Here we can see different objects. A car is an object. A person is an object. A book is an object. Each object has data and actions. Objects can interact with each other. Together, they make a program.



What's next?

We will set up the development environment, write our first program "Hello, World!", look at compilation and execution, and study the basics of classes and objects.



5 Basic Concepts of OOP

Object-Oriented Programming (OOP) is a programming paradigm based on representing a program as a set of interacting objects, each of which is an instance of a certain class, combining state (data) and behavior (methods).

Main concepts of OOP

- **Object** — a concrete entity in the problem domain that has a state (data) and behavior (methods). Each object is an instance of a class.
- **Class** — a template (description) of an object that defines the structure of the state (fields) and the behavior (methods). Multiple objects of this class can be created.
- **State (data)** — the current values of an object's fields that describe its properties.
- **Behavior (methods)** — functions of a class that define actions that an object can perform and may change its state.
- **Message** — a request to an object to perform a method. Objects interact by exchanging messages.

Key principles of OOP

1. **Encapsulation** — combining data and methods inside a class and hiding the internal implementation from external code.
2. **Inheritance** — creating new classes based on existing classes to reuse code and extend functionality.
3. **Polymorphism** — the ability to use objects of different classes through a common interface.
4. **Abstraction** — highlighting the essential characteristics of an object and hiding unnecessary details.



Why is this important?

OOP increases the modularity, extensibility and maintainability of programs. The class model helps to organize code, making the system more flexible and easier to understand.

Structure of a Java class

A class in Java combines fields (state) and methods (behavior).

```
public class Car {  
    // 1. Fields (state of the object)  
    private String brand;    // car brand  
    private int year;        // production year  
  
    // 2. Constructor – special method to initialize the object  
    public Car(String brand, int year) {  
        this.brand = brand;    // assign field values  
        this.year = year;  
    }  
  
    // 3. Method (behavior of the object)  
    public void start() {  
        System.out.println("The engine is started!");  
    }  
}
```

Car
- brand: String - year: int
+ Car(brand: String, year: int) + start(): void

- **private** — access modifier: the field is accessible only inside the class.
 - **this** — reference to the current object, used to distinguish class fields from local variables.
- The constructor is called when creating a new object and initializes its initial state.

Creating and using an object

To work with a class, we need to create an object using the `new` operator and call its methods.

```
public class Main {  
    public static void main(String[] args) {  
        // 1. Creating an object of class Car  
        Car myCar = new Car("Toyota", 2020);  
  
        // 2. Access to object fields  
        // (through methods, not directly to fields)  
  
        // 3. Sending a message to the object –  
        // calling the start() method  
        myCar.start();  
    }  
}
```

Program output:

```
>_ The engine is started!
```

Why is this code useful?

- Demonstrates the basic structure of a class in Java.
- Shows how to create an object and call a method.
- Clearly illustrates the interaction between an object and a class.
- Demonstrates encapsulation: fields are private, external code works with the object only through methods (constructor and methods).

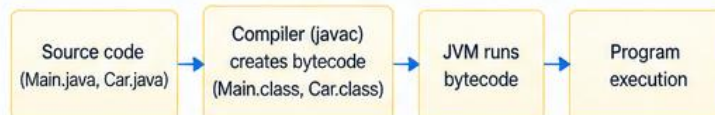
Practical note

In the following chapters, we will look at OOP mechanisms in more detail: inheritance, polymorphism, abstraction and interfaces, and we will also apply them in practice when developing more complex systems.

Here we have a class **Car**. It has two fields: **brand** and **year**. The fields are private. The constructor creates a new **Car** object. And this means the current object.

Private means we cannot access them directly from another class.

From a class to a working program (in short)



Java is a compiled language with an intermediate representation (bytecode) that is executed by the JVM. This provides platform independence ("write once, run anywhere").

Programming Paradigms (Programming



Definition. A programming paradigm is a set of fundamental ideas and principles that define the way we think about programming and how we build software systems.

A paradigm answers the following questions:

- How to divide a problem into parts?
- How to organize data?
- How do these parts interact and behave?
- What language features and tools are used to express these ideas?

Here we can see **five programming paradigms**.

They are different ways of programming.

Java can use **different paradigms**.

The main paradigms in Java are **imperative and OOP**.

Java also supports **functional programming**.

Main programming paradigms

Paradigm	Main idea	Key features	Example languages
1. Imperative	A program is a sequence of commands that change the state of data.	• step-by-step execution • changing variables (state) • loops, conditions, assignments	C, Pascal, Java (basic), C++
2. Object-Oriented (OOP)	A program is a system of interacting objects, each with a state (data) and behavior (methods).	• encapsulation • inheritance • polymorphism • abstraction	Java, C++, C#, Python, Swift
3. Functional (FP)	A program is a composition of functions. The main focus is on using functions and avoiding changing data.	• functions as first-class values • immutable data • higher-order functions • function composition	Haskell, Lisp, Scala, Kotlin, Java (Stream API), F#
4. Logical	A program is a set of facts and logical rules. The result is obtained by logical inference.	• facts and rules • automatic inference • unification and backtracking	Prolog, Datalog
5. Declarative	Specify what result you want, not how to achieve it. The system finds the solution.	• focus on the desired result • no explicit control flow (loops, conditions)	SQL, HTML, CSS

Comparison of approaches: calculate the sum of array elements

Imperative style (step-by-step)

```
int sum = 0;
for (int x : arr) {
    sum += x;
}
return sum;
```

In the imperative style, we clearly write the steps to solve the problem.

Functional style (what to do)

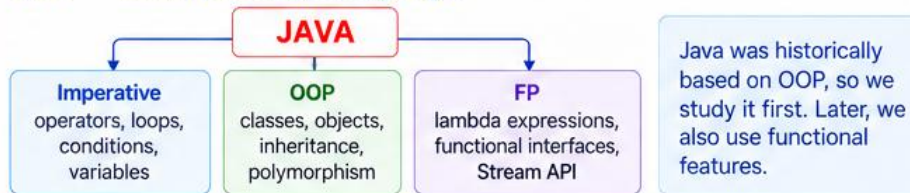
```
return Arrays.stream(arr)
    .sum();
```

In the functional style, we describe what we want to do — calculate the sum using built-in operations (data flow).

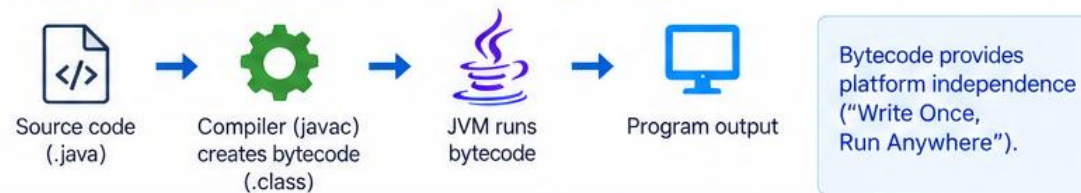
Why is it useful to understand different paradigms?

- ✓ Allows you to choose the best approach for a problem.
- ✓ Improves code quality: modularity, readability, and testability.
- ✓ Functional ideas complement OOP: fewer errors due to immutability and parallelism.
- ✓ Modern languages (Java 8+) support multi-paradigm programming.

Java — multi-paradigm language



How a Java program is created and executed (in short)



Conclusion

A paradigm is a way of thinking about how to develop programs. Understanding different paradigms helps us build flexible, reliable and easily maintainable systems and solve problems more effectively.

Key takeaways

- ✓ There are several programming paradigms.
- ✓ Java supports multiple paradigms, mainly OOP.
- ✓ The choice of paradigm depends on the problem and the context.

Object-Oriented Programming (OOP)



1. Definition of OOP

Object-oriented programming (OOP) is a programming methodology in which a program is considered as a collection of interacting objects, each of which is an instance of a certain class.

2. Object

An object is an instance of a class that has a certain state (data) and behavior (methods).

For example:

car object → **state:** brand, speed
behavior: drive, stop



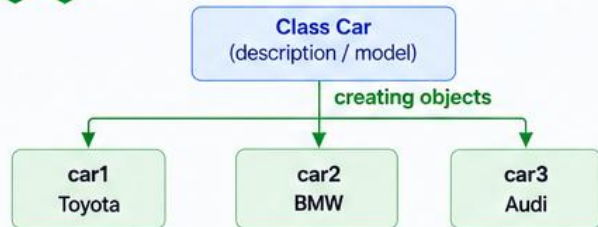
3. Class

To create multiple similar objects, we define a class — a data type that describes the structure (fields) and behavior (methods) of objects that can be created from it.



4. Class and object

A class defines a structure, and an object is a specific instance of this structure.



A class defines the structure of objects. Objects are concrete instances of a class with their own state (field values).

Structure of a Java class (example)

```

class Car {
    // Fields (state of the object)
    String brand;    // car brand
    int speed;       // speed

    // Constructor (initialization of the object)
    Car(String brand) {
        this.brand = brand;    // assign field value
        this.speed = 0;        // initial speed
    }

    // Method (behavior of the object)
    void start() {
        System.out.println("The engine is started!");
    }
}
  
```

Fields (brand, speed) store the object's state (data).
The constructor `Car(String brand)` initializes the object's fields.
The method `start()` describes the action that the object can perform.

Creating and using an object

```

public class Main {
    public static void main(String[] args) {
        // 1. Creating an object of class Car
        Car myCar = new Car("Toyota");

        // 2. Accessing the object's state
        System.out.println("Brand: " + myCar.brand);
        System.out.println("Speed: " + myCar.speed);

        // 3. Calling a method on the object
        myCar.start();
    }
}
  
```



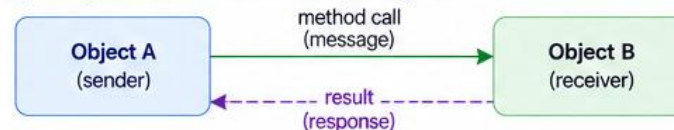
Program output:

```

Brand: Toyota
Speed: 0
The engine is started!
  
```

5. Program as a system of objects

Objects do not exist in isolation. During program execution, they interact by calling each other's methods and passing data.



OOP organizes a program as a system of interacting objects whose state is represented by data (fields) and behavior by methods.

Here we can see a Java class and an object.

On the left, we have the class Car.

The class has fields, a constructor, and a method.

The fields store data about the car.

The constructor creates a new object.

The method defines an action.

On the right, we create a Car object.

Then we can access its data.

And finally, we call the method start.

What to tell students (in short)

First, we explain that programs can be built in different ways. One of the key paradigms is OOP. Its main idea is to represent a program as a set of objects. Each object has a state (data) and behavior (methods). To define the structure and behavior of similar objects, we create a class. Then we will study classes, objects and their interaction in the Java language in more detail.



8 Object and Class (Object and Class)



Object and class are two fundamental concepts of OOP.

A class is a template (blueprint) that describes the structure and behavior of objects of a certain type.

An object is a specific instance of a class with its own state (data) and behavior, capable of performing actions.



Important!

A class describes what data and behavior objects of a certain type have.

Objects are real instances created based on the class.

1. What is a class?

A class is a user-defined data type that:

- contains a set of fields (data) that represent the state of objects;
- contains a set of methods (functions) that define the behavior of objects.

A class does not take up memory. It is a template based on which objects can be created.

2. What is an object?

An **object** is an instance of a class.

Each object has:

- its own state (field values);
- the ability to perform methods (behavior).

Object car1



State:

brand = "Toyota"
year = 2020

Behavior (methods):

start(), stop(), info()

3. Example of class and creating objects in Java

Class (template)

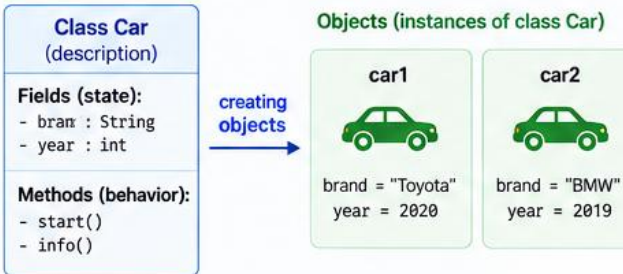
```
class Car {  
    // Fields (state)  
    String brand;  
    int year;  
  
    // Methods (behavior)  
    void start() {  
        System.out.println("Engine started!");  
    }  
  
    void info() {  
        System.out.println(brand + " " + year);  
    }  
}
```

Creating and using objects

```
public class Main {  
    public static void main(String[] args) {  
        Car car1 = new Car();  
        car1.brand = "Toyota";  
        car1.year = 2020;  
  
        car1.start();  
        car1.info();        // Toyota 2020  
  
        Car car2 = new Car();  
        car2.brand = "BMW";  
        car2.year = 2019;  
  
        car2.info();        // BMW 2019  
    }  
}
```

The same class can be used to create any number of objects (car1, car2, ...), each of which has its own field values and can call methods independently.

4. Relationship between a class and an object

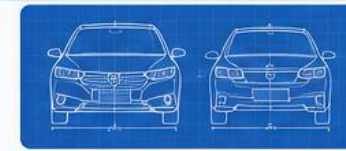


A class defines a common structure.
Objects are concrete instances with their own state (field values).

What to cover at this stage (in short)

- ✓ What a class and an object are.
- ✓ How the class description (blueprint) differs from an object.
- ✓ What state an object has.
- ✓ How to create objects from a class.
- ✓ What object state (data) and behavior (methods) are.

Here we can see a class and objects.
The class Car is a template.
It describes a car.
It has fields and methods.
On the right, we create different Car objects.
For example, Toyota, BMW, and another car.
Each object has its own data.
But all objects use the same class.



Important?

to describe abstractions of real-world objects.
program modular and scalable.

s own state and behavior,
code development and reuse.

Example: Class and Objects (Example: Class and Objects)



Let's look at a simple example: the **Student** class and objects of this class.

A **class** describes what data (state) and what actions (behavior) students have.

Objects are concrete students with their own field values.



Important!

A class is a blueprint (template).
Objects are created based on the class
and have their own state in memory.

1. Problem description

We need to describe students.

Each student has:

- name (**name**),
- age (**age**),
- average grade (**gpa**).

A student can:

- display information about themselves,
- increase their average grade.



2. Student class (template)

The class defines the structure (fields) and behavior (methods) of Student objects.

```
class Student {
    // Fields (state)
    String name;    // student name
    int age;        // age
    double gpa;     // average grade

    // Constructor
    Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        this.gpa = gpa;
    }

    // Method (behavior)
    void showInfo() {
        System.out.println("Name: " + name +
            ", Age: " + age + ", GPA: " + gpa);
    }

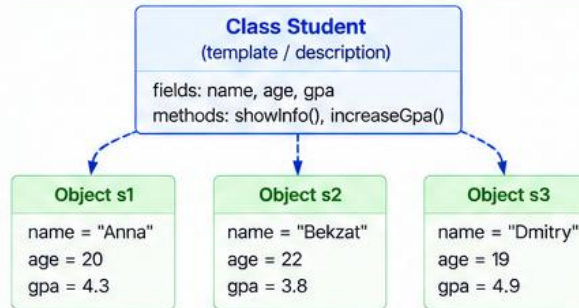
    void increaseGpa(double delta) {
        gpa += delta;
        if (gpa > 5.0) gpa = 5.0;    // maximum grade = 5
    }
}
```

The constructor `Student(...)` is called when an object is created and initializes its fields with the initial values.
The methods `showInfo()` and `increaseGpa(...)` define the behavior of the object.

3. Creating objects of the Student class

Based on the Student class, we create objects with specific data.

```
Student s1 = new Student("Anna", 20, 4.3);
Student s2 = new Student("Bekzat", 22, 3.8);
Student s3 = new Student("Dmitry", 19, 4.9);
```



4. Using objects (calling methods)

```
s1.showInfo();    // display information about the student
s1.increaseGpa(0.2); // increase grade by 0.2
s1.showInfo();    // information after the change
System.out.println();
s2.showInfo();
s3.showInfo();
```

Program output (example):

```
Name: Anna, Age: 20, GPA: 4.3
Name: Anna, Age: 20, GPA: 4.5
Name: Bekzat, Age: 22, GPA: 3.8
Name: Dmitry, Age: 19, GPA: 4.9
```

5. What happens?

Step		Description
1	Define the class	The Student class describes what data students have and what actions they can perform.
2	Create objects	The new Student(...) operation allocates memory and creates a specific object with its own field values.
3	Call methods	Methods perform actions on the object's data. Each object has its own state.
4	State changes	After calling increaseGpa(...), the value of the gpa field for the object changes.
5	Objects are independent	Each object has its own field values. Changes in one object do not affect others.

6. State of objects during execution

Initial state		After the change
s1: name = "Anna" age = 20 gpa = 4.3 s2: name = "Bekzat" age = 22 gpa = 3.8	s1.increaseGpa(0.2) Only the gpa field	s1: name = "Anna" age = 20 gpa = 4.5 s2: name = "Bekzat" age = 22 gpa = 3.8

Now let's look at a simple example.

Here we have a Student class.

The class is a template for students.

It has three fields: name, age, and GPA.

It also has methods.

Then we create three Student objects: Anna, Bekzat, and Dmitry.

Each object has its own data.

For example, Anna is 20 and her GPA is 4.3.



Key takeaways

- A class defines the structure and behavior of objects.
- An object stores its own field values and performs methods.
- Each object has its own independent state in memory.
- OOP helps to create modular, clear and easily reusable programs.



Remember

- Class = description (blue)
- Object = real instance
- Fields = data (state)
- Methods = actions (behavior)

10 Main Principles of OOP (Main Principles of OOP)



OOP is based on several fundamental principles that help us create reliable, flexible, extensible and maintainable programs. Three key principles of OOP: **Encapsulation, Inheritance and Polymorphism.**



Important!

These principles do not exist separately from each other. They complement and strengthen each other, allowing us to effectively model real-world entities and their behavior.

1 Encapsulation

Idea: combine data (state) and methods (behavior) inside a class and hide the internal implementation from external access.

Goal: protect data from unauthorized access and modifications, and provide a controlled interface for working with the object.

```
class BankAccount {  
    private double balance; // private field  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Explanation:

- balance is a private field.
- It can be changed only through methods.
- This hides the internal implementation and protects the object's state.



Result

The object manages its own state and provides safe access to the data.



Real-world analogy

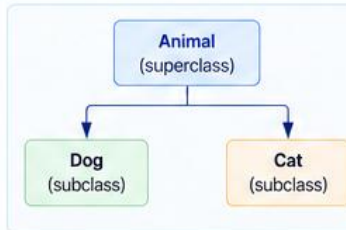
A bank account (data) can be opened and closed only using specific methods, not by direct access.

2 Inheritance

Idea: one class (subclass) and inherit fields and methods from another class (superclass) and extend or modify them.

Goal: reuse code, create a hierarchy of classes and express the "is-a" relationship.

```
class Animal {  
    void eat() { System.out.println("Eats"); }  
}  
  
class Dog extends Animal {  
    void bark() { System.out.println("Barks"); }  
    @Override  
    void eat() { System.out.println("Dog eats"); }  
}  
  
class Cat extends Animal {  
    void meow() { System.out.println("Meows"); }  
}
```



Explanation:

- Dog and Cat inherit from Animal.
- Dog overrides eat() and adds bark().
- Cat adds its own method meow().
- The original class is not modified.



Real-world analogy

A dog and a cat are animals. They have common features (eat), but also their own specific behavior (bark, meow).

3 Polymorphism

Idea: one and the same interface (method) can have different implementations in different classes.

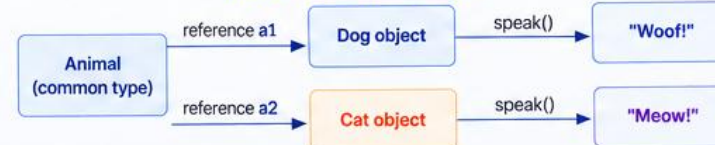
Goal: work with objects of different classes through a common type (superclass or interface).

```
class Animal {  
    void speak() { System.out.println("Some sound"); }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() { System.out.println("Woof!"); }  
}  
  
class Cat extends Animal {  
    @Override  
    void speak() { System.out.println("Meow!"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Dog(); // same type, different objects  
        Animal a2 = new Cat();  
  
        a1.speak(); // Woof!  
        a2.speak(); // Meow!  
    }  
}
```

Explanation:

- The same method (speak) has different implementations.
- When called on objects of different classes, the appropriate version of the method is executed (runtime polymorphism).
- This allows writing flexible and extensible code.

How polymorphism works



How OOP principles work together

- 1 **Encapsulation** keeps data safe and hides implementation.
- 2 **Inheritance** allows building a hierarchy of classes and reusing code.
- 3 **Polymorphism** allows working with objects of different classes through a common interface.
- 4 Together they help create modular, understandable, extensible and easy-to-maintain programs.

Summary table

Principle	Key idea	Main benefit
Encapsulation	Hide data and provide controlled access	Data protection, integrity of object state
Inheritance	Reuse properties and methods of an existing class	Code reuse, hierarchy of classes
Polymorphism	One interface — multiple implementations	Flexibility, extensibility, working with different objects



Key takeaways

- ✓ Encapsulation — hides data to protect it.
- ✓ Inheritance — extends functionality and allows code reuse.
- ✓ Polymorphism — one interface, multiple implementations.
- ✓ These principles are the foundation of OOP in modern programming languages, including Java.

11 Functional Programming (FP)



Definition

Functional programming (FP) is a programming paradigm in which programs are built from functions.

Functions do not change data — they create new values.



Important!

- ✓ No data modification (immutable).
- ✓ Functions are pure computations.
- ✓ Code becomes simpler, shorter, and safer.

Main idea



A function takes input data and returns a result.

It does not change the input data.

Example

Example: adding two numbers
(in Java)

```
// function
static int add(int a, int b) {
    return a + b;    // returns a new result
}

// usage
int result = add(3, 5);    // result = 8
System.out.println(result);
```

Key principles of FP



Functions as first-class citizens

Functions can be stored in variables, passed as arguments and returned from other functions.



Immutability (no changes)

Data is not modified. New values are created instead.



Pure functions

For the same input, the function always returns the same output.



Function composition

Programs are built by combining small functions together.

Comparison: FP vs OOP

Functional Programming (FP)

Object-Oriented Programming (OOP)



Does not modify data
(works with new values).

Data can be modified.



Functions are the main tool.

Classes and objects
are the main tool.



Simpler to test and understand.
Fewer side effects.

Closer to the real world.
Convenient for large systems.

Another example: doubling numbers

```
// Imperative way (modifies a list)
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
for (int i = 0; i < list.size(); i++) {
    list.set(i, list.get(i) * 2);
}
// result: [2, 4, 6]

// Functional way (does not modify the original list)
List<Integer> list = List.of(1, 2, 3);
List<Integer> doubled = list.stream()
    .map(n -> n * 2)
    .toList();
// doubled: [2, 4, 6]    list remains [1, 2, 3]
```

When to use FP?

- ✓ Data processing and transformations.
- ✓ Parallel computations.
- ✓ Working with streams (streams).
- ✓ When reliability and testability are important.



Key idea:

Functional programming is about functions, immutable data, and creating new results without changing the original data.



Same input — same output
Predictable, simple and powerful!

12 Basic Ideas of Functional Programming (FP)



Definition

Functional programming (FP) is a paradigm based on simple but powerful ideas.

A program is a set of functions.

Functions do not change data; they create new values.



Important!

- ✓ No data modification (immutable).
- ✓ Functions are pure computations.
- ✓ Code becomes simpler, shorter, and safer.

6 Key Ideas of Functional Programming

1 Functions — the core

$f(x)$

A program consists of functions. Functions can be passed to other functions and return results.

Example:

```
map(), filter(),  
reduce(), ...
```

2 Pure functions



The same input always gives the same output. No hidden changes or side effects.

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

3 Immutability

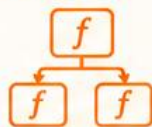


Data is not changed after creation. New values are created, and old ones remain unchanged.

Example:

```
List<Integer> list =  
    List.of(1, 2, 3);  
// list cannot be modified
```

4 Higher-order functions



Functions can take other functions as arguments and return functions as results.

Example:

```
List<Integer> result =  
    numbers.stream()  
        .map(x -> x * 2)  
        .toList();
```

5 Function composition



Complex problems are solved by combining small functions.

Example:

```
numbers.stream()  
    .filter(x -> x > 0)  
    .map(x -> x * 2)  
    .toList();
```

6 Working with data as streams



Data is processed step by step. Operations like map, filter, reduce, and others are used to work with data streams.

Example:

```
int sum = numbers.stream()  
    .filter(x -> x > 0)  
    .reduce(0, Integer::sum);
```

Simple example: sum of a list of numbers

Imperative approach (modifies data)

```
int sum = 0;  
for (int n : new int[]{1, 2, 3, 4}) {  
    sum += n;  
}  
System.out.println(sum); // 10
```

Functional approach (does not modify data)

```
int sum = Arrays.stream(  
    new int[]{1, 2, 3, 4})  
    .sum();  
System.out.println(sum); // 10
```

What happens?

[1, 2, 3, 4]

↓

sum()

↓

10 (result)

Why is FP useful?



Safety

No data modification means fewer errors.



Simplicity

Code is easier to understand and test.



Reliability

Pure functions work as expected.



Parallelism

Functions are independent and can be executed in parallel.

Where is FP used?



Java (Stream API)

Functional style in modern Java (8+).



Web development

Data processing, APIs, microservices.



Data analysis

Stream processing of large datasets.



Other languages

Scala, Kotlin, Python, JavaScript and more.



Key takeaway:

Functional programming is about working with functions and immutable data to create new values in a clear, simple, and reliable way.



From data —
to new insights!

13 OOP vs FP — Comparison of Approaches

Object-Oriented Programming vs Functional Programming



OOP and **FP** are two different ways to write programs.
Each approach has its own strengths and is suitable for different situations.



Important!

- ✓ Different ideas — the same goal: solve problems.
- ✓ You can use both approaches.
- ✓ Choose the approach that fits your task best.

OOP (Object-Oriented Programming)	Criterion	FP (Functional Programming)
 Main idea A program is built from objects and classes.	 1. Main idea	 Main idea A program is built from functions.
 Data Data is stored inside objects. It can be modified.	 2. Data	 Data Data is immutable (not changed). New values are created instead.
 Modifying data Objects can be modified.	 3. Data modification	 Data modification No modification of data. Only new data is created.
 How it works The state of an object changes during program execution.	 4. Execution	 How it works Functions take input and return output without side effects.
 Code structure Code is organized around objects and their interactions.	 5. Structure	 Code structure Code is organized around functions and their composition.
 Suitable for Large systems, games, applications with complex objects.	 6. When to use	 Suitable for Data processing, computations, problems where reliability and simplicity are important.

OOP — a model of the real world

We have objects (person, car, house).
They have properties and can perform actions.



Main takeaway:

Both approaches help write good programs.
Choose the one that best fits your task!

FP — a set of functions

We have functions.
They take data as input and return a result.



Remember:

OOP = objects and state

FP = functions and immutable data

14 Java Programming Language — What is Java?



Java is a modern programming language.

It is simple to learn, reliable and works on different devices.

! IMPORTANT!

- ✓ Java is easy to read and write.
- ✓ Java programs are safe and reliable.
- ✓ Write once — it works everywhere!

WHAT IS JAVA?



Programming language

Used to create applications and programs.



Reliable and secure

Checks for errors and protects your data.



Object-oriented

Works with objects and classes.



Developed by Sun Microsystems

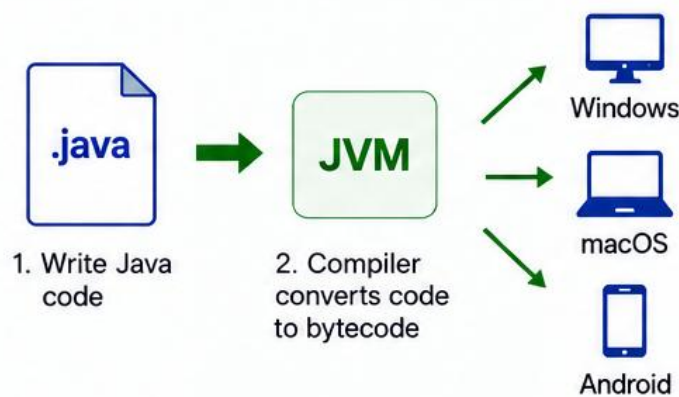
In 1995.



Widely used

In web, mobile applications, games, large systems and more.

HOW DOES JAVA WORK? (WRITE ONCE, RUN ANYWHERE)



✓ **JVM (Java Virtual Machine)** runs the program on any platform.

ADVANTAGES OF JAVA



Platform independence

Works on Windows, macOS, Linux, Android and many others.



Simplicity

Easy to read, write and understand.



Security

Protects from many errors and viruses.



Reliability

Used in important systems all over the world.



Large community

Many examples, libraries and support.

WHERE IS JAVA USED?



Web applications
(websites, services)



Mobile applications
(Android)



Corporate systems
(banks, companies)



Games



Cloud applications
and systems

ABOUT JAVA



Java is a programming language that helps people create applications that work fast, safely and on different devices.

Write once — run anywhere!



KEY IDEA: Java is a simple, reliable and universal programming language used by millions of developers around the world.





Java has many strong features that make it popular and convenient for developers.

! IMPORTANT!

- ✓ Java is a simple, reliable and secure language.
- ✓ It is suitable for small and very large projects.
- ✓ Write once — it works everywhere!

1 Simple



Easy to read, write and understand.
Similar to English.

2 Platform Independent (Write Once, Run Anywhere)



The code is compiled to bytecode and runs on any platform where a JVM is available.

3 Object-Oriented



Based on classes and objects.
Makes development easier and helps to organize code.

4 Reliable and Secure



Checks for errors during compilation and runtime.
Protects from viruses and crashes.

5 Automatic Memory Management



Has a garbage collector.
You do not need to delete memory manually.

6 Multithreading



Allows many tasks to run at the same time.
Improves program performance.

7 High Performance



JVM optimizes the code during execution.
Programs run fast.

8 Distributed



Easy to create network and distributed applications.

9 Big Standard Library



Java has a large standard library for many tasks.
You write less code.

10 Popular



Used by millions of developers worldwide.
Many examples, tutorials and support.



KEY IDEA:

Java is a modern, convenient and powerful programming language that is suitable for many tasks and works on different devices and systems.



16 Structure of a Java Program

A simple Java program has three main parts: class, main() method, statements.



Java program = Class + main() method + Statements

The program starts from the main() method.



IMPORTANT!

- ✓ Every program must have one class.
- ✓ Every program must have a main() method.
- ✓ Statements are the instructions.

EXAMPLE OF A SIMPLE JAVA PROGRAM

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         // This is a statement (instruction)
4         System.out.println("Hello, World!");
5         int number = 10;
6         System.out.println(number);
7     }
8 }
```

1 CLASS

- Every Java program must have a **class**.
- A class is the container for the program.
- In this example: **HelloWorld** is the class name.

2 main() METHOD

- main() is the starting point of the program.
- **public static void main(String[] args)** is required.
- The program runs the code inside this block { ... }.

3 STATEMENTS

- Statements are instructions.
- Examples:
 - print text: `System.out.println("Hello, World!");`
 - create a variable: `int number = 10;`
 - use variable: `System.out.println(number);`

HOW IT WORKS

Program starts
from **main()**

Statements
are executed
one by one

Program
finishes

KEY POINTS



Class

Holds the program.
Only one public class
per file.



main()

The starting point.
Must be exactly
written.



Statements

Do the work.
Many statements
inside main().



Semicolon (;)

Most statements
end with a semicolon.

OUTPUT (RESULT)



First line is text.
Second line is the value
of the variable number.
This is the result of the program.



SUMMARY:

A Java program has a class.
Inside the class there is a main() method. Inside main() there are statements.

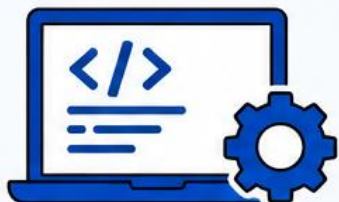


JDK, JRE and JVM — differences

JDK

Java Development Kit

Complete toolkit for developing and running programs



- ✓ Includes JRE + development tools
- ✓ Compiler (javac)
- ✓ Debugger (jdb)
- ✓ Other tools (jar, javadoc, etc.)

Used by:
developers
to create programs

JRE

Java Runtime Environment

Environment for running Java programs



- ✓ Includes JVM + standard class libraries
- ✓ Does not contain development tools (no compiler)
- ✓ Needed only to run ready-made programs

Used by:
end users
to run programs

JVM

Java Virtual Machine

Virtual machine for Java — “engine” that executes code

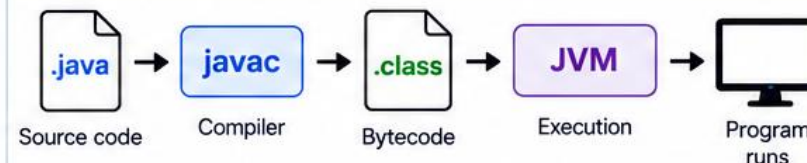


- ✓ Executes Java bytecode
- ✓ Provides platform independence (Write Once, Run Anywhere)
- ✓ Manages memory, threads, security and more

Used by:
the internal component of JRE
(end users do not use it directly)

COMPARISON	JDK	JRE	JVM
Purpose	Development + running	Only running	Executing bytecode
What it includes	JRE + development tools	JVM + class libraries	Executes bytecode, manages memory and resources
Who uses it	Developers	End users	Used inside JRE

HOW DOES IT WORK?

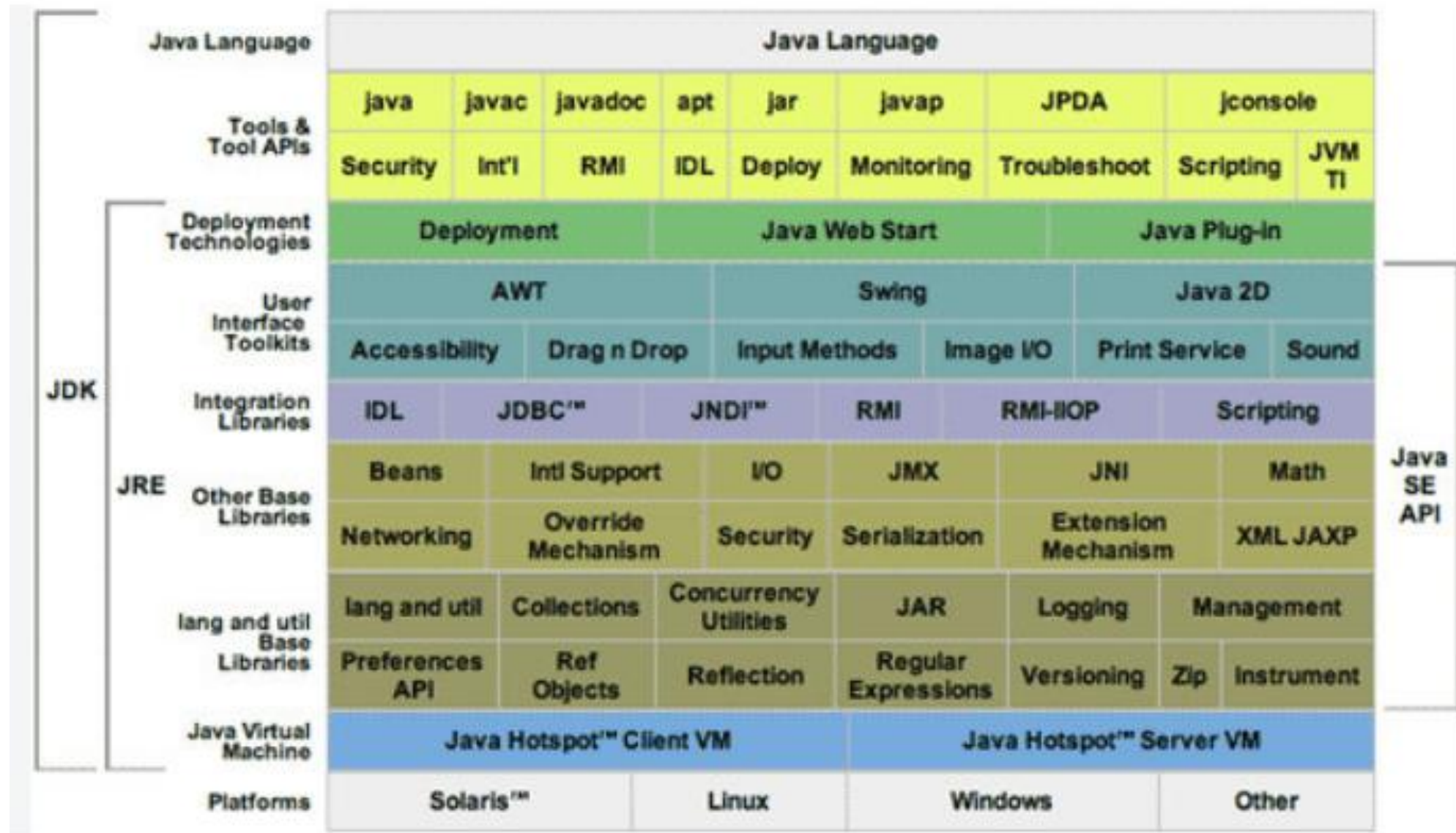


KEY IDEA:

JDK contains everything for development.

JRE is needed only for running programs.

JVM executes bytecode and makes Java platform independent.

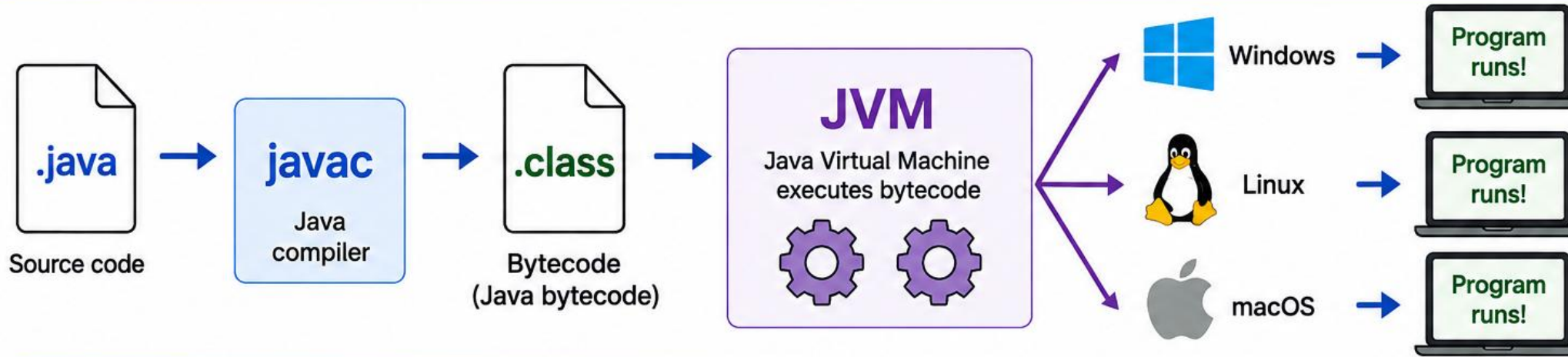


Java Platform Independence

Write Once, Run Anywhere



The same Java program can run on different operating systems **without any changes** and without recompilation.



Why is this possible?

1. A Java program is compiled not into machine code of a specific operating system, but into bytecode (.class).
2. The same bytecode works on all platforms.
3. The JVM on each operating system translates the bytecode into machine code and runs the program.

Advantages

- ✓ Portability
- ✓ Time saving
- ✓ Reliability



Write Once, Run Anywhere (WORA) — write a program once, run it everywhere!

Simple Java Application – 'Hello World'

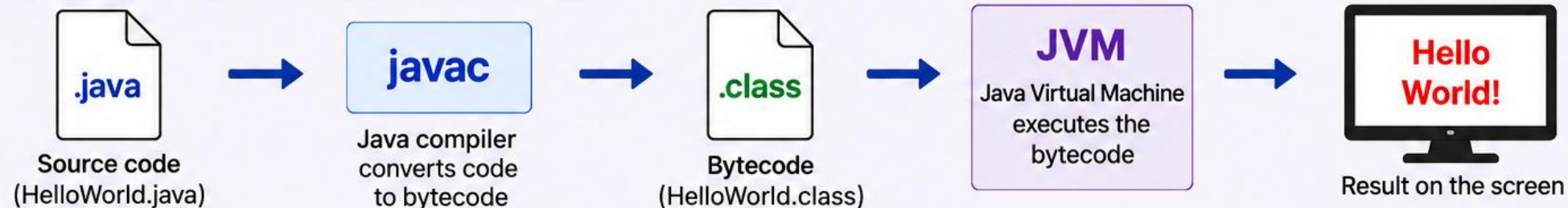
1. Source Code (HelloWorld.java)

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World!");  
4     }  
5 }
```

2. What does this code do?

- 1 The program is called **HelloWorld**.
- 2 The **main** method is the entry point. Execution of the program starts from here.
- 3 The command **System.out.println("Hello World!");** prints the text **Hello World!** to the screen.
- 4 After running the program, we see on the screen:
Hello World!

3. How does the program work?



4. Commands to run



1) Compile: `javac HelloWorld.java`

2) Run: `java HelloWorld`

5. Result

```
> java HelloWorld  
Hello World!
```



Program runs correctly!



Hello World — the first Java program. It shows that everything works correctly!

Structure of a Java Program

class, main() method, statements

1. EXAMPLE OF A JAVA PROGRAM

```

1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello World!");
4         System.out.println("Java is interesting!");
5     } // end of main() method
6 } // end of class

```

2. PARTS OF A JAVA PROGRAM

1

CLASS

- A class is a container for the program.
- The program name and class name are usually the same (HelloWorld).

2

main() METHOD

- Every Java program starts from the main() method.
- It is the entry point — execution starts from here.

3

STATEMENTS (INSTRUCTIONS)

- Statements are the commands of the program.
- They perform actions and produce a result.

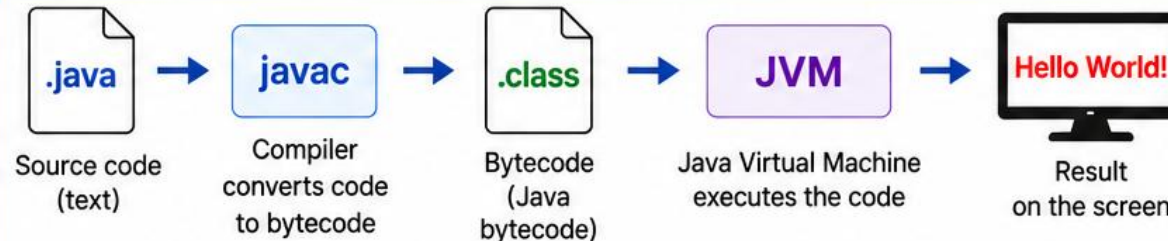


RULES:

1. The class name must start with a capital letter.
2. The main() method must be written exactly as:

```
public static void main(String[] args)
```
3. Each statement (instruction) must end with a semicolon ;

3. HOW DOES THE PROGRAM WORK?



SUMMARY:

A Java program consists of a **class**, the **main()** method and **statements**.

The **main()** method is the starting point of the program.

Statements perform actions and produce a result.

Write once — run anywhere!
(Write Once, Run Anywhere)

The main() Method

The entry point of a Java program

SYNTAX

```
public static void main(String[] args) {
    // here are the program statements
}
```

public

Accessible from outside (the JVM)

static

Can be called without creating an object

void

Does not return any value

String[]

Array of strings (command-line arguments)

args

Name of the argument array

WHAT IS main() ?



main() is the entry point of any Java program.



When we run a program, the JVM looks for the **main()** method and starts execution from there.



Execution of **main()** begins the work of the program.



All statements inside **main()** are executed in order from top to bottom.

EXAMPLE

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

This line prints text to the screen.

HOW DOES IT WORK?



Source code (text)



Compiler converts code to bytecode



Bytecode (Java bytecode)



Java Virtual Machine starts execution



Result on the screen



KEY POINTS:



Every Java program must have a **main()** method.



The class name and file name should be the same.



Without a correct **main()** method, the program will not run.



main() is the start of our program!

IDE for Java Development

IntelliJ IDEA / Eclipse, etc.



An IDE (Integrated Development Environment) is a program that helps you **write**, **run** and **debug** Java programs.

1. POPULAR IDES FOR JAVA



IntelliJ IDEA

Very smart and convenient.
Popular for professional development.



Eclipse

Free and lightweight.
Good for learning and small projects.



VS Code

Simple and fast.
Needs a Java extension (Extension Pack for Java).

2. WHAT DOES AN IDE DO?



Write code

Syntax highlighting, code completion.



Run the program

One click — and the program runs.



Debug

Find and fix errors.



Manage the project

Files, libraries, build tools, etc.

3. COMPARISON OF IDES

IDE	Free	Easy for beginners	Useful features	Performance	Best for
 IntelliJ IDEA	Community Edition (free)	★★★★☆	★★★★★	★★★★★	Large and medium projects, professional development
 Eclipse	Yes	★★★★☆	★★★★☆	★★★★☆	Learning, small and medium projects
 VS Code (+ Java Extension)	Yes	★★★★☆	★★★★☆	★★★★☆	Quick small projects, web and cloud development



SUMMARY:



An IDE makes development **faster and more convenient**.



IntelliJ IDEA is powerful and smart, **Eclipse** is free and lightweight, **VS Code** is simple and fast.



Choose the IDE that suits you best!

Creating a Java Project

структура простого проекта

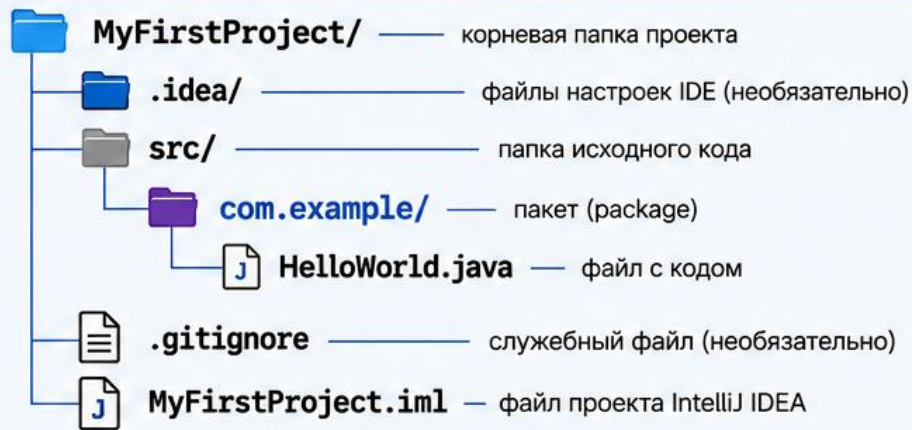


Java-проект — это папка с файлами, в которой мы пишем, запускаем и управляем программой. IDE (например, IntelliJ IDEA или Eclipse) помогает создавать структуру проекта автоматически.

1. ЧТО ТАКОЕ ПРОЕКТ?

- ✓ Проект хранит все файлы вашей программы.
- ✓ Имеет определенную структуру папок.
- ✓ Легко добавлять новые классы и файлы.
- ✓ Удобно запускать, отлаживать и управлять кодом.

2. СТРУКТУРА ПРОЕКТА (ПРИМЕР)



3. ЧТО НАХОДИТСЯ В ФАЙЛАХ?

 src/	Здесь находится ваш код (пакеты и классы).
 com.example/	Пакет (package). Он группирует классы. Имя пакета может быть любым.
 HelloWorld.java	Файл класса. В нём пишется код программы (класс и метод main()).
 Другие файлы	Служебные файлы (например, .gitignore) используются IDE для настроек и управления проектом.

4. КАК СОЗДАТЬ ПРОЕКТ (В INTELLIJ IDEA)



ИТОГ:



Проект — это папка с файлами.



Код находится в папке src/ (в пакетах).



Файлы организованы в пакеты (packages).



IDE помогает создавать, запускать и управлять проектами.

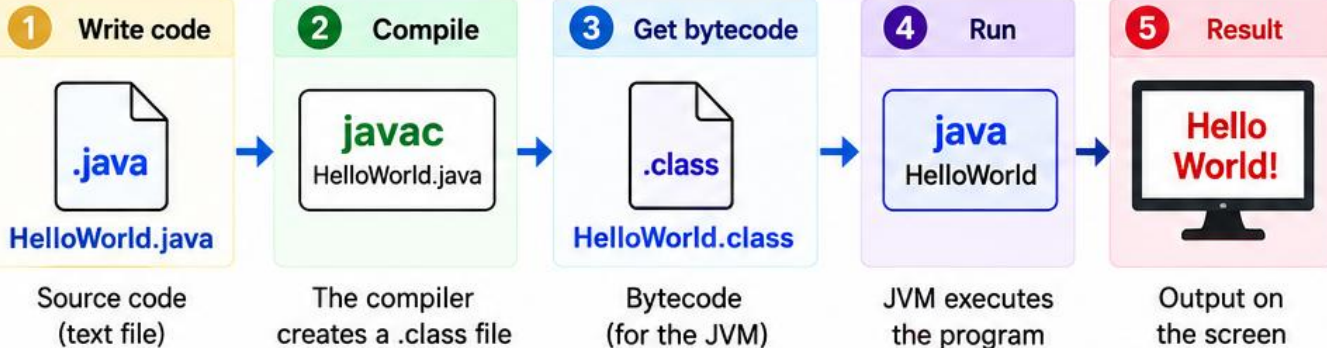
Compilation and Execution

javac and java



To run a Java program, we first compile it with **javac**, and then run it with **java**.
The compiler converts the **source code (.java)** into **bytecode (.class)**, and the JVM executes the bytecode.

1. HOW DOES IT WORK?



2. MAIN COMMANDS

javac – Java compiler

```
javac HelloWorld.java
```

- Compiles the `.java` file into a `.class` file.
- If there are errors in the code, the `.class` file will not be created.

java – run a program

```
java HelloWorld
```

- Starts the program using the JVM.
- Specify the class name without `.class`.

3. EXAMPLE IN THE TERMINAL (Command Line)

```
C:\MyProject> javac HelloWorld.java
# 1. Compile
# A new file
# HelloWorld.class
# is created

C:\MyProject> java HelloWorld
# 2. Run
# Program output
Hello World!
```

4. IMPORTANT THINGS TO KNOW



The file name and class name must match.
For example: file `HelloWorld.java` — class `HelloWorld`.



The class must contain a `main()` method.
It is the entry point of the program.



`.java` is the source code, `.class` is the bytecode.
The JVM executes the `.class` file.



SUMMARY:

Write code
(.java)



Compile
javac



Get bytecode
(.class)



Run
java



Program
works!

26 Classes in Java — basic structure of a class



A class is a blueprint (template) for creating objects.
A class can contain **fields (data)** and **methods (behavior)**.

! Important!

- ✓ A class name starts with the keyword **class**.
- ✓ Class names are written with a capital letter.
- ✓ A file can contain at most one **public** class.

BASIC STRUCTURE OF A CLASS

```
public class MyClass {  
    // 1. Fields (data)  
    int number;  
    String name;  
  
    // 2. Method (behavior)  
    public void sayHello() {  
        System.out.println("Hello!");  
    }  
  
    // 3. main() - program entry point  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj.number = 10;  
        obj.name = "Java";  
        obj.sayHello();  
    }  
}
```

1 CLASS
Defines the type of objects.

2 FIELDS (DATA)
Store the data of an object.

3 METHODS (BEHAVIOR)
Define what an object can do.

4 main() METHOD
The entry point where the program starts execution.

CREATING AND USING AN OBJECT



1 Create an object from the class
MyClass obj = new MyClass();



2 Work with fields (data)
obj.number = 10;
obj.name = "Java";



3 Call methods (behavior)
obj.sayHello();

OUTPUT (RESULT)



The following will be displayed on the screen:
Hello!



Key takeaway: A class is a blueprint. An object is a real instance of that blueprint. An object has data (fields) and can perform actions (methods).



Class
(blueprint)



Object
(real instance)

27 Creating Objects — using `new` and object creation



To use a class, we need to create an object.
The keyword **`new`** allocates memory and creates an object.



Important!

- ✓ An object is created using the keyword **`new`**.
- ✓ Each object is a separate copy.
- ✓ After creation, we can work with the object's fields and methods.

Example in Java

```
1 class Car {  
2     String color;  
3     int speed;  
4 }  
5  
6 public class Main {  
7     public static void main(String[] args) {  
8         // 1. Create an object of class Car  
9         Car myCar = new Car();  
10  
11        // 2. Work with the object  
12        myCar.color = "Red";  
13        myCar.speed = 120;  
14    }  
}
```

1 Creating an object

`Car myCar = new Car();`

A new object of class Car is created.
A reference to the object is stored in the variable `myCar`.

2 Using the object

We access the object's fields using the dot operator (`.`).
We assign values to the fields `color` and `speed`.

3 Each object is unique

If we create multiple objects, each one will have its own independent field values.

How it works



1 Define the class

We create a class (e.g., Car), which describes the structure (fields and methods).



2 Create an object using `new`

The expression `new Car()` allocates memory and creates an object.



3 Reference to the object

The variable (`myCar`) stores a reference (address) to this object.



4 Work with the object

We can read and modify fields and call the object's methods.

A simple analogy



A class is a blueprint of a house.
An object is a real house built from that blueprint.
The **`new`** keyword is like the command: "Build a house!"

Result (output)



An object is created.
We assigned values to its fields.



Key takeaway: To use a class, we create an object using **`new`**.
An object is a real instance of a class.



Class
(blueprint)



Object
(real instance)

Fields and Methods

the state and behavior of an object



Every object in Java has a **state (fields)** and **behavior (methods)**.
Fields store data, and methods define what the object can do.

IMPORTANT!

- ✓ Fields describe what data an object has.
- ✓ Methods describe what an object can do.
- ✓ We call methods on an object.

1. EXAMPLE OF A JAVA CLASS

```

1  class Car {
2      // fields (state) - data of the object
3      String color;
4      int speed;
5
6      // methods (behavior) - actions of the object
7      void start() {
8          System.out.println("Car starts");
9      }
10     void stop() {
11         speed = 0;
12         System.out.println("Car stops");
13     }

```

Fields (state)

Methods (behavior)

2. FIELDS (STATE)



Fields store data.
This is the **state** of an object.

Examples of fields:

- color — the car's color
- speed — the car's speed

Fields can have different types:
int, double, String, boolean, etc.

3. METHODS (BEHAVIOR)



Methods perform actions.
This is the **behavior** of an object.

Examples of methods:

- start() — starts the car
- stop() — stops the car

Methods can take parameters
and return values (**void, int**, etc.).

4. HOW IT WORKS

1



Create an object

```
Car myCar = new Car();
```

2



Set field values

```
myCar.color = "Red";
myCar.speed = 120;
```

3



Call methods

```
myCar.start();
myCar.stop();
```

4



The object performs actions

The car starts
and then stops.

5. SIMPLE ANALOGY



STATE (FIELDS)

- Color
- Speed
- Amount of fuel

BEHAVIOR (METHODS)

- Start
- Drive
- Stop

A car has a **state** (data) and can perform **actions** (methods).

6. USAGE IN A PROGRAM

```

Car myCar = new Car();    // create an object
myCar.color = "Red";      // set the state
myCar.speed = 120;        // set the state
myCar.start();            // call a method
myCar.stop();             // call a method

```

Car starts
Car stops



KEY IDEA:

Fields — store the data (**state**) of an object.
Methods — perform actions (**behavior**) of an object.

Fields
(state)

+

Methods
(behavior)

=

Object

29 Console Input and Output — System.out, Scanner



In Java, we can display data on the screen (**output**) and read data from the user (**input**) using the **Scanner class**.



Important!

- ✓ **System.out** is used for output to the console.
- ✓ **Scanner** is used for reading input from the user.
- ✓ Close the Scanner at the end of the program.

Example Program

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5
6         System.out.print("Enter your name: ");
7         String name = sc.nextLine(); // read a line of text
8
9         System.out.print("Enter your age: ");
10        int age = sc.nextInt(); // read an integer
11
12        System.out.println("Hello, " + name +
13                           "! You are " + age + " years old.");
14        sc.close(); // close the Scanner
15    }
16 }
```

1. Output — System.out

We use **System.out** to display text on the screen.

- **System.out.print()** — prints without a new line.
- **System.out.println()** — prints with a new line.

2. Input — Scanner

The **Scanner** class reads data from the user through the keyboard.

- **Scanner sc = new Scanner(System.in);** — creates a Scanner for reading input.
- **nextLine()** — reads a line of text.
- **nextInt()** — reads an integer.
- **close()** — closes the Scanner.

How it works (step by step)

- 1 The program displays a prompt on the screen.



- 2 The user enters data using the keyboard.



- 3 The program reads the data and stores it.



- 4 The program displays the result on the screen.



Example Input and Output (on the screen)

Example user input:

Enter your name: **Anna**
Enter your age: **20**



Example program output:

Hello, Anna!
You are 20 years old.

Common Scanner Methods

Method	What it does	Example input
nextLine()	Reads a line of text (string)	Hello
nextInt()	Reads an integer	123
nextDouble()	Reads a floating-point number	3.14

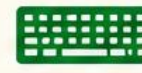


Key takeaway: **System.out** displays data on the screen.
Scanner reads data from the user.



System.out

displays data
on the screen



Scanner

reads data
from the user



The program
interacts with
the user!

Practice for Students (in class)

Topic: Java Code Conventions + Summary



Write Java programs.

Use **Scanner** for input and **System.out** for output.



Goals:

- Practice taking input (Scanner) and output (System.out).
- Learn to calculate sums.
- Follow Java naming conventions.

1 Problem 1. Sum of the first N integers

The user enters a positive integer N .
Calculate the sum of the first N natural numbers.

$$S = 1 + 2 + 3 + \dots + N$$

Example:

If $N = 5$, then $S = 1 + 2 + 3 + 4 + 5 = 15$

Input:

Enter N: 5

Output:

Sum = 15



Hint: you can use a loop to add numbers from 1 to N .

2 Problem 2. Sum of a series of fractions

The user enters a positive integer N .
Calculate the sum of the series:

$$S = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{N}$$

Example:

If $N = 4$, then

$$S = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = 2.0833$$

Input:

Enter N: 4

Output:

Sum = 2.0833



Hint: use the **double** type for the sum.

3 Problem 3. Alternating series

The user enters a positive integer N .
Calculate the sum of the series:

$$S = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \dots + (-1)^{N+1} \frac{1}{N}$$

Example:

If $N = 4$, then

$$S = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} = 0.5833$$

Input:

Enter N: 4

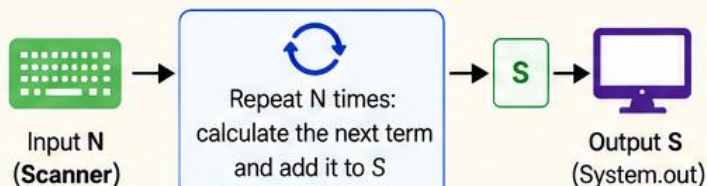
Output:

Sum = 0.5833



Hint: the sign of the term depends on the position: if k is odd — "+", if k is even — "-".

General idea (for all problems)



General requirements

- ✓ Use meaningful variable names.
- ✓ Read input using **Scanner**.
- ✓ Print results using **System.out.println()**.
- ✓ Follow Java code **conventions**.
- ✓ Test your program with different values of N .

★ Remember!



- N is a positive integer.
- Use the **int** type for N .
- Use the **double** type for sums with fractions.
- Print the sum with reasonable precision (e.g., up to 4 decimal places after the point).



Goal: write simple Java programs that calculate the sum of series terms step by step and output the result.

31 Programming Exercises for Students (in class)

Topic: Geometric Problems



Write programs in **Java**.

Use **Scanner** for input and **System.out** for output.



Goal of the exercise:

- Practice input of data (Scanner) and output (System.out).
- Learn to solve geometric problems.
- Follow Java naming conventions.

4 Problem 4. Circle

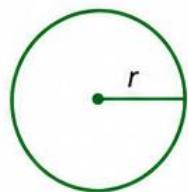
The user enters the radius **r**.

Find the area of the circle and the circumference.

Formulas:

$$S = \pi r^2 \text{ — area of the circle}$$

$$L = 2\pi r \text{ — circumference}$$



Example:

Enter r: 5

Area = 78.54

Circumference = 31.42



Hint (Java):

```
double area = Math.PI * r * r;
```

```
double length = 2 * Math.PI * r;
```

5 Problem 5. Right Triangle

The user enters the lengths of the legs **a** and **b**.

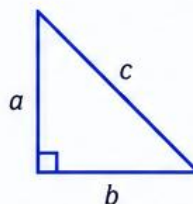
Find the hypotenuse, area, and perimeter.

Formulas:

$$c = \sqrt{a^2 + b^2} \text{ — hypotenuse}$$

$$S = \frac{ab}{2} \text{ — area}$$

$$P = a + b + c \text{ — perimeter}$$



Example:

Enter a: 3

Enter b: 4

Hypotenuse = 5

Area = 6

Perimeter = 12



Hint (Java):

```
double c = Math.sqrt(a * a + b * b);
```

```
double area = a * b / 2;
```

```
double perimeter = a + b + c;
```

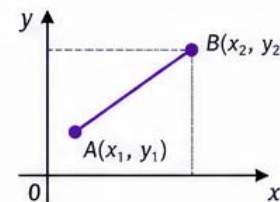
6 Problem 6. Distance Between Points

The user enters the coordinates of two points:

A(x₁, y₁) and **B(x₂, y₂)**. Find the distance between them.

Formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Example:

Enter x1: 1

Enter y1: 2

Enter x2: 4

Enter y2: 6

Distance d = 5



Hint (Java):

```
double dx = x2 - x1;
```

```
double dy = y2 - y1;
```

```
double d = Math.sqrt(dx * dx + dy * dy);
```

General idea of the solution



Input data
(Scanner)



Calculate
using formulas



Output result
(System.out)

General requirements

- ✓ Use meaningful variable names.
- ✓ Input data using **Scanner**.
- ✓ Output the result using **System.out.println()**.
- ✓ Use **double** for real numbers.
- ✓ Follow Java Code Conventions.



Remember!

- π = **Math.PI**
- Square root: **Math.sqrt(x)**
- Print real numbers with 2–4 decimal places after the point.



Goal: write simple programs that compute geometric quantities using formulas and output the results.

32 Programming Exercises for Students (in class)

Topic: Geometric Transformations of a Triangle



Write programs in **Java**.

Use **Scanner** for input and **System.out** for output.



Goal of the exercise:

- Practice input of data (Scanner) and output (System.out).
- Learn to perform geometric transformations of figures.
- Follow Java naming conventions.

1 Problem 1. Translation (Shift)

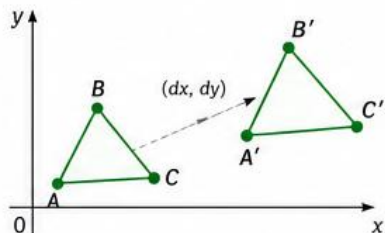
The user enters the coordinates of the triangle vertices $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$ and the translation vector (dx, dy) .

Find the new coordinates of the vertices A' , B' , C' after the translation.

Formulas:

$$\begin{aligned}x' &= x + dx \\ y' &= y + dy\end{aligned}$$

(for each vertex)



Example:

$A(1, 1)$ $B(4, 1)$ $C(2, 3)$ $\rightarrow$ $A'(4, 3)$ $B'(7, 3)$
 $dx = 3, dy = 2$ $C'(5, 5)$



Hint (Java):

```
double newX = x + dx;
double newY = y + dy;
// do this for all 3 vertices
```

2 Problem 2. Scaling (Resize)

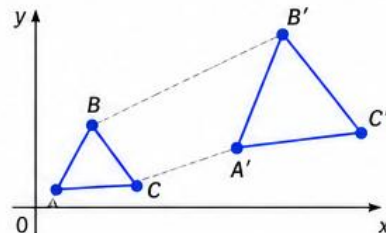
The user enters the coordinates of the triangle vertices $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$ and the scaling coefficients S_x and S_y .

Find the new coordinates of the vertices A' , B' , C'

Formulas:

$$\begin{aligned}x' &= x \cdot S_x \\ y' &= y \cdot S_y\end{aligned}$$

(for each vertex)



Example:

$A(1, 1)$ $B(4, 1)$ $C(2, 3)$ $\rightarrow$ $A'(2, 2)$ $B'(8, 2)$
 $S_x = 2, S_y = 2$ $C'(4, 6)$



Hint (Java):

```
double newX = x * scaleX;
double newY = y * scaleY;
// do this for all 3 vertices
```

Note:

- $S_x > 1 \rightarrow$ enlarge
- $S_x = 1 \rightarrow$ no change
- $0 < S_x < 1 \rightarrow$ reduce

3 Problem 3. Rotation

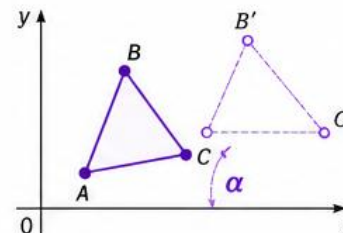
The user enters the coordinates of the triangle vertices $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$ and the rotation angle α (in degrees) around the origin $(0, 0)$.

Find the new coordinates of the vertices A' , B' , C' .

Formulas:

$$\begin{aligned}x' &= x \cos \alpha - y \sin \alpha \\ y' &= x \sin \alpha + y \cos \alpha\end{aligned}$$

(for each vertex)



Example:

$A(1, 0)$ $B(0, 2)$ $C(2, 0)$ $\rightarrow$ $A'(0, 1)$ $B'(-2, 0)$ $C'(0, 2)$
 $\alpha = 90^\circ$



Hint (Java):

```
double rad = Math.toRadians(angleDegrees);
double newX = x * Math.cos(rad) - y * Math.sin(rad);
double newY = x * Math.sin(rad) + y * Math.cos(rad);
// do this for all 3 vertices
```

Note:

- Use radians in `Math.sin` and `Math.cos`.
- `Math.toRadians()`.

General workflow



Input data
(Scanner)



Calculate using
formulas



Output result
(System.out)

General requirements

- ✓ Use meaningful variable names.
- ✓ Read input using **Scanner**.
- ✓ Perform all calculations according to the formulas.
- ✓ Output the result using `System.out.println()`.
- ✓ Use **double** for real numbers.
- ✓ Follow Java Code Conventions.

Formulas for reference

Translation	$x' = x + dx$	$y' = y + dy$
Scaling	$x' = x \cdot S_x$	$y' = y \cdot S_y$
Rotation	$x' = x \cos \alpha - y \sin \alpha$	$y' = x \sin \alpha + y \cos \alpha$



Goal: write simple programs that perform translation, scaling and rotation of a triangle and output the new coordinates of the vertices.